

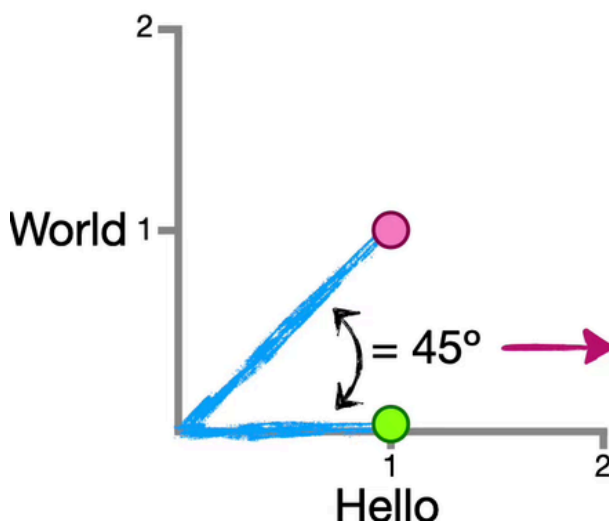
Vector Database

made by vijay singh

The **Cosine Similarity** is a relatively easy to calculate metric that can tell us how similar or different things are.

	Hello	World
Hello, World! →	1	1
Hello! →	1	0

...and thus, the **Cosine Similarity** between the phrases, “**Hello, World!**” and “**Hello!**” is $\cos(45^\circ) = 0.71$.

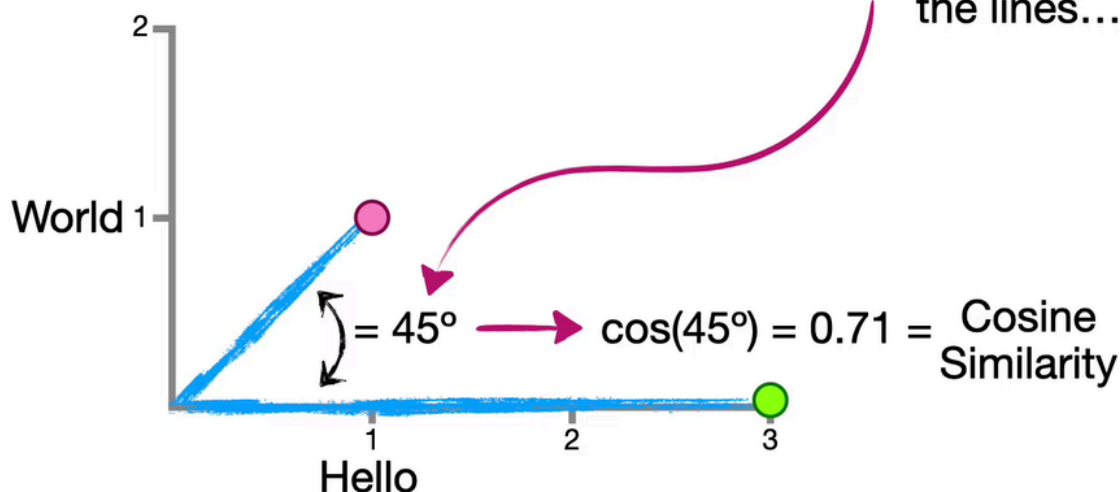


$$\cos(45^\circ) = 0.71 = \text{Cosine Similarity}$$



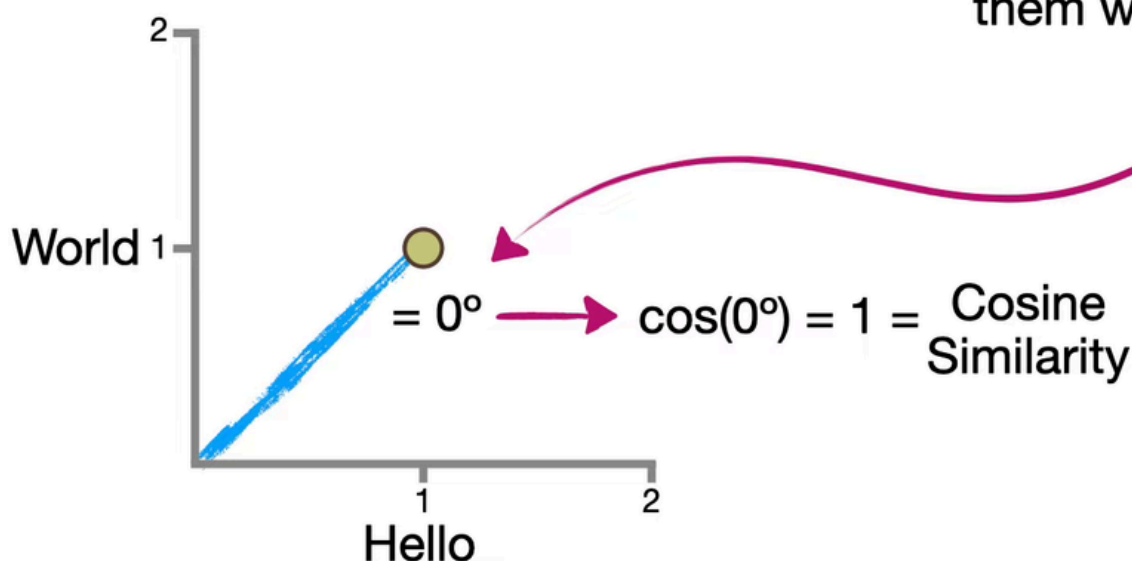
	Hello	World
Hello, World! →	1	1
Hello Hello Hello! →	3	0

In other words, the **Cosine Similarity** is determined entirely by the **angle** between the lines...



	Hello	World
Hello, World! →	1	1
Hello, World! →	1	1

So, when both phrases are exactly the same, the angle between them will be **0°**...



Summarize:

Two phrases nothing in common → cosine similarity 0

Exactly same → cosine similarity 1

Some overlap → cosine similarity between 0 and 1

$$\text{Cosine Similarity} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

sigma → short hand notation for adding up things

i → index we are working on

n → number of different words in the phrases

A, B → refer to phrases



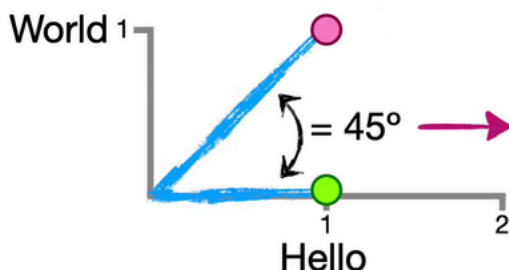
A = Hello, World! →

B = Hello! →

	Hello	World
A	1	1
B	1	0

...which is the same value
we got when we took the
cos(45°).

$$\text{Cosine Similarity} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} = \frac{(1 \times 1) + (1 \times 0)}{\sqrt{1^2 + 1^2} \sqrt{1^2 + 0^2}} = 0.71$$



$\cos(45^\circ) = 0.71 = \text{Cosine Similarity}$



A = I love Troll 2! →

B = I love Gymkata! →

I	Love	Troll	2	Gymkata
1	1	1	1	0
1	1	0	0	1

In summary, the **Cosine Similarity** is a relatively easy to calculate metric that can tell us how similar or different things are.

$$\begin{aligned}
 \text{Cosine Similarity} &= \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \\
 &= \frac{(1 \times 1) + (1 \times 1) + (1 \times 0) + (1 \times 0) + (0 \times 1)}{\sqrt{1^2 + 1^2 + 1^2 + 1^2 + 0^2} \sqrt{1^2 + 1^2 + 0^2 + 0^2 + 1^2}} \\
 &= 0.58
 \end{aligned}$$

Euclidean Distance:

$$\text{Euclidean Distance} = |X - Y| = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

X: Array or vector X

Y: Array or vector Y

x_i: Values of horizontal axis in the coordinate plane

y_i: Values of vertical axis in the coordinate plane

n: Number of observations



Cosine Similarity

Cosine similarity measures how similar two vectors are by computing the cosine of the angle between them, focusing on direction rather than magnitude.

In short:

Same direction = high similarity (even if sizes differ)

Euclidean Distance

Euclidean distance measures the straight-line distance between two vectors in space, capturing how far apart they are in terms of absolute position.

In short:

Closer points = more similar

Question: When do we use cosine similarity and euclidean distance?

Cosine Similarity vs Euclidean Distance

When to use what — and WHY

Big Picture (one-liner)

- **Cosine similarity** → *Direction matters, magnitude doesn't*
- **Euclidean distance** → *Exact position & magnitude matter*

1. Cosine Similarity

What it measures

Angle between vectors

“Do these two vectors point in the same direction?”

”

Why it's powerful

Cosine similarity **ignores scale**.

Example:

[100, 200, 300]

[10, 20, 30]

Same direction → cosine = **1**

Meaning is same, but size different.

When to use Cosine Similarity

1. Text Search & NLP

- Semantic search
- Question answering
- Chatbots
- Document similarity

Why?

Long document vs short document can mean same thing.

Used in:

- Vector databases
- LLM embeddings

2. Recommendation Systems

- YouTube
- Netflix
- Spotify
- Amazon

Why?

User interest direction matters, not how much activity they had.

3. High-Dimensional Data

- 384 / 768 / 1536-dim embeddings
- Magnitude becomes noisy
- Angle stays meaningful

4. Sparse Data

- TF-IDF
- Bag of Words
- Word embeddings

When NOT to use cosine

- When **distance in space matters**
- When **absolute values have meaning**

Key properties

Property	Value
Scale sensitive?	No
Range	-1 to 1
Focus	Direction
Robust to vector length	Yes

2. Euclidean Distance

What it measures

Straight-line distance between points

“How far apart are these two points in space?”

Why it's useful

It captures **absolute difference**.

Example:

LocationA: (10,10)

LocationB: (13,14)

Distance tells real spatial gap.

When to use Euclidean Distance

1. Physical Space / Geometry

- Maps
- GPS
- Robotics
- Computer vision

2. Clustering (K-Means)

Why?

Cluster based on closeness in space.

K-Means assumes:

- Euclidean distance
- Spherical clusters

3. Numerical Features with Same Scale

- Height, weight, age (after normalization)
- Sensor values
- Pixel values

4. Low-Dimensional Data

- 2D, 3D, 10D
- Euclidean works well

Problems with Euclidean in ML

- Sensitive to scale
- High dimensions → **distance concentration problem**
- Magnitude dominates meaning

Key properties

Property	Value
Scale sensitive?	Yes
Range	$0 \rightarrow \infty$
Focus	Position
High-dim friendly?	No

Side-by-Side Comparison Table

Aspect	Cosine Similarity	Euclidean Distance
Measures	Angle	Distance
Scale sensitivity	No	Yes
Best for	Text, embeddings	Geometry, clustering
High-dim data	Excellent	Poor
Used in vector DBs	Yes	Rare

4. Real-World Examples

YouTube Search

- Uses **cosine similarity**
- Why?
 - “web dev course” vs “full stack dev”
 - Same meaning, different words

Amazon Products

- Cosine → product similarity
- Euclidean → price difference, size difference

Spotify

- Cosine → taste similarity
- Euclidean → loudness, tempo

5. Important Hidden Truth

Cosine similarity + normalized vectors = Euclidean distance

If vectors are **L2 normalized**:

Euclidean distance $\propto 1 - \text{cosine similarity}$

Euclidean distance $\propto 1 - \text{cosine similarity}$

That’s why many systems:

- Normalize embeddings

- Use dot product (faster)

6. Decision Rule (Gold)

Ask yourself

Does magnitude matter?

- No → **Cosine similarity**
- Yes → **Euclidean distance**

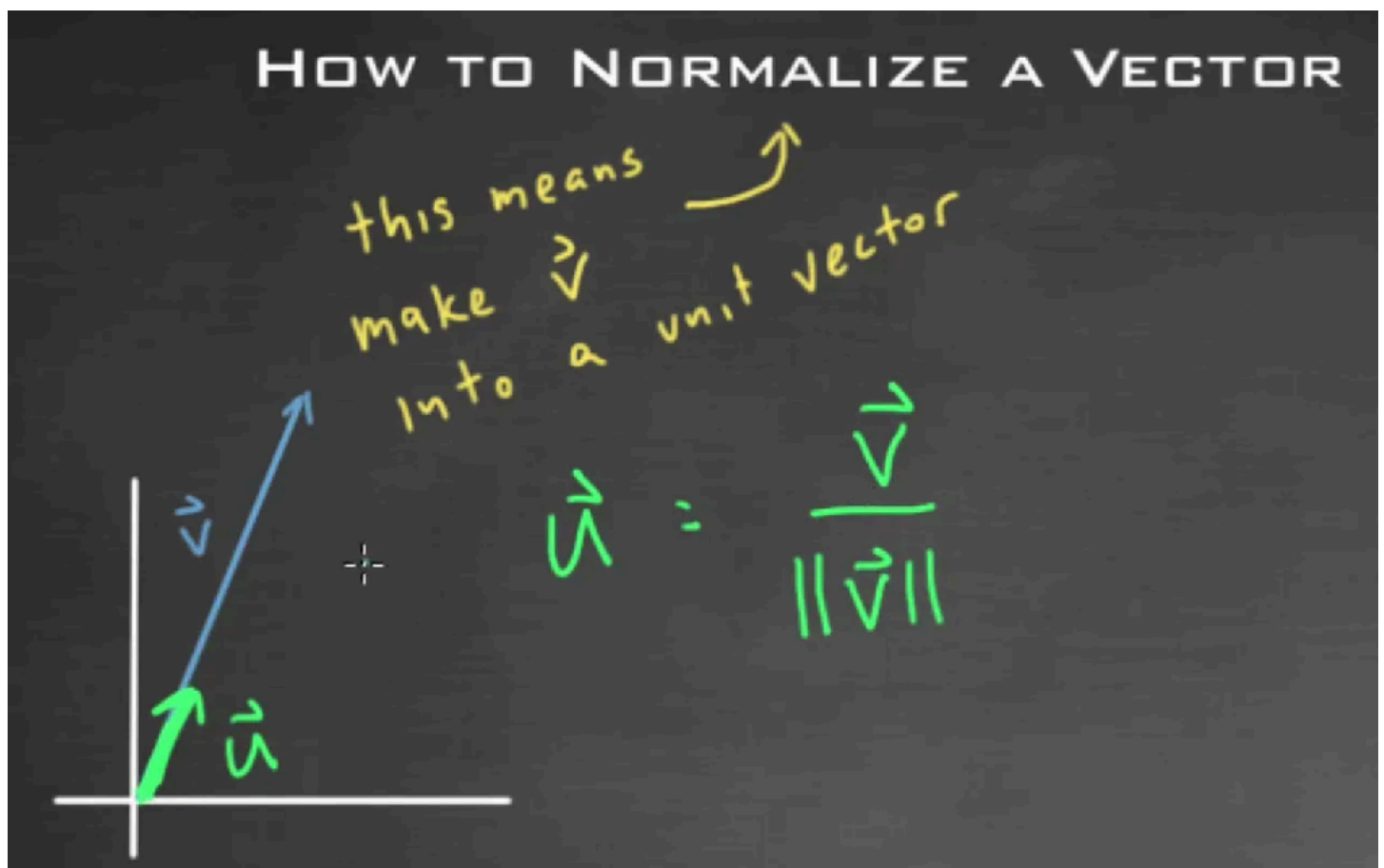
7. Quick Mental Model

- **Cosine** = “Are we thinking in same direction?”
- **Euclidean** = “How far apart are we standing?”

TL;DR (Perfect revision lines)

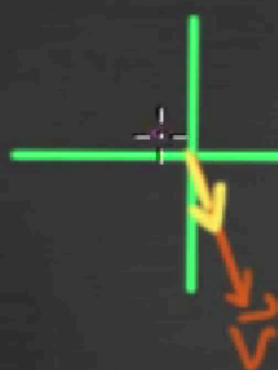
- Use **cosine similarity** for **text, embeddings, recommendations**
- Use **Euclidean distance** for **spatial, numeric, geometric problems**
- Vector databases mostly use **cosine or dot product**
- High dimensions → cosine wins

Question: What is normalization of a vector?



Ex: $\vec{v} = \langle 4, -9 \rangle$

$$\|\vec{v}\| = \sqrt{16 + 81}$$



$$\|\vec{v}\| = \sqrt{97}$$

$$\vec{u} = \frac{1}{\sqrt{97}} \langle 4, -9 \rangle$$

$$\vec{u} = \left\langle \frac{4}{\sqrt{97}}, \frac{-9}{\sqrt{97}} \right\rangle$$

$$\|\vec{u}\| = 1$$

What is a Vector Database?

A vector database is a specialized database designed to store, index, and search vectors efficiently using similarity measures like cosine similarity or Euclidean distance.

In simple words:

A DB that finds “most similar” data instead of exact matches.

Why normal databases fail here

Traditional DB:

```
SELECT * WHERE text= "web dev course";
```

Vector DB:

Find top-K vectors closest to my query vector

One-line intuition (very important)

- **Vector:** *Meaning encoded as numbers*
- **Vector DB:** *Google for meanings, not keywords*

HOW THINGS ARE SEARCHED (FROM SCRATCH)

1. What does “search” really mean?

At the most basic level:

Search = find things that are similar to what the user wants

Example:

- You type “**milk**”
- System should show:
 - milk packet
 - toned milk
 - almond milk
 - maybe curd / butter

So the problem is:

How do computers understand “similar”?

2.Old-school search (keywords)

Before AI, search worked like this:

"milk" → matchword "milk" in product name

Problems:

- “dairy” ≠ “milk”
- “almond milk” may not contain exact word
- Meaning is lost

So keyword search breaks.

3.New idea: represent meaning as numbers

Instead of words, we convert **meaning** → **numbers**

This is called an **embedding**.

4.What is an embedding? (super beginner)

Imagine a magic machine:

"milk" → [0.12,0.78,0.34, ...]

"curd" → [0.11,0.80,0.31, ...]

"chips" → [0.90,0.02,0.10, ...]

Key idea:

Similar meanings → similar number patterns

This vector is just:

- A point in space
- Meaning encoded as direction

5.Step-by-step: how search ACTUALLY happens

Let’s walk through what happens when you type “milk”.

STEP 1: Items are prepared (offline)

Before users even search:

Milk packet → embedding → store

Curd → embedding → store

Butter → embedding → store

Chips → embedding → store

All vectors are stored in a **vector database**.

STEP 2: User types a query

You type:

"milk"

STEP 3: Query is also converted to a vector

"milk" → [0.13,0.79,0.33]

Same embedding model is used.

STEP 4: Compare query vector with item vectors

Now system asks:

“Which stored vectors are closest to this query vector?”

This is **similarity search**.

6.How do we measure “closeness”?

Two common methods:

1.Cosine similarity (direction)

- Angle between vectors
- Ignores magnitude
- Most common in semantic search

2.Euclidean distance (actual distance)

- Straight-line distance
- Sensitive to magnitude

In practice:

Cosine similarity + normalized vectors

7. Why not compare with ALL items?

If there are:

- 1,000,000 products

Brute force = too slow

So instead...

8. The BIG IDEA: “Search smart, not everywhere”

Instead of checking every vector:

Search only in nearby regions

This is where **vector databases** come in.

9. What does a vector DB do?

A vector DB:

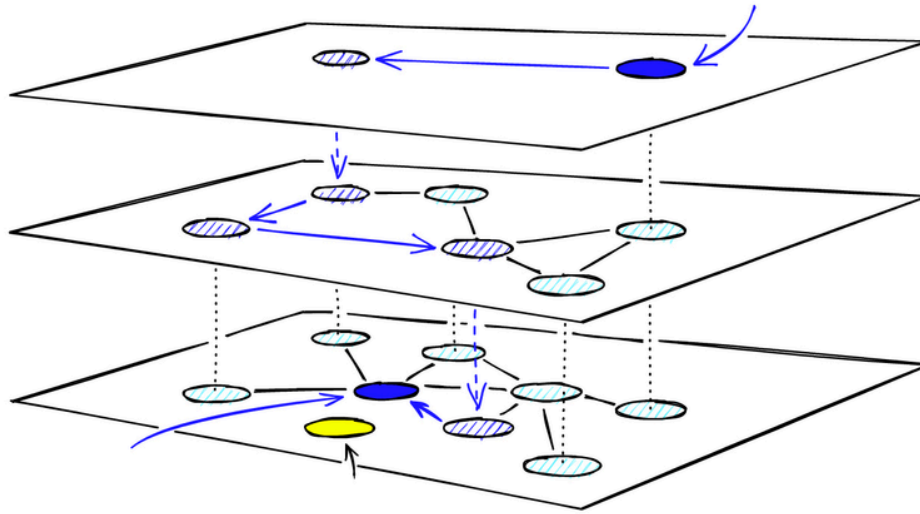
- Stores vectors
- Builds a **smart index**
- Finds nearest neighbors FAST

It does **NOT** sort vectors linearly.

How fast search actually works (intuition)

Vector Search

Hierarchical Navigable Small Worlds (HNSW)



- Vectors are connected to similar vectors
- Query enters graph
- Jumps from close node → closer node
- Stops when no closer neighbor exists

This is called **Approximate Nearest Neighbor (ANN)**.

Why “approximate” is okay?

Because:

- Exact search = slow
- Approximate search:
 - 99% accurate
 - 100x faster

Recommendation systems prefer **speed over perfection**.

Final full pipeline (end-to-end)

Products → Embeddings → Vector DB (indexed)

User query

↓

Embedding



ANNSearch



Top-Ksimilar items



Apply filters (price, availability)



Show results

Important misconceptions (CLEAR THESE)

- ✗ Sorting vectors
- ✗ Binary search
- ✗ Using 1D logic on multi-D data

Why?

- Vectors live in **space**, not on a number line.

Real-world examples

App	What vectors represent
Blinkit	Product meaning
Amazon	Product + user intent
Netflix	Movies + taste
Spotify	Songs + mood
Google	Web page meaning

One-line memory hook

Search = embedding + nearest neighbors in vector space

Beginner-level summary

- Words → vectors
- Similar meaning → nearby vectors
- Vector DB → fast neighbor lookup
- ANN → avoids brute force
- Sorting/binary search → wrong tool